

Final Project Proposal

Class: COMP 5300
Name: Jegannarayanan Sivakasiulaganathan
Student ID: 02197006
Email: Jegannarayanan_Sivakasiulaganathan@student.uml.edu

1 Overview

My final project proposal is to create a Python AST to LLVM MLIR IR Object pipeline. The idea is to leverage the library mlir-hs, which consists of API calls to create MLIR IR objects. Reference: <https://github.com/google/mlir-hs>

2 LLVM MLIR IR objects

This section briefly looks at what MLIR IR objects look like, and why they are important. In LLVM, MLIR is a intermediate representation between high-level source code and LLVMMIR:

Source Code (Python, C, C++, etc.) $\rightarrow$ MLIR $\rightarrow$ LLVMMIR $\rightarrow$ Machine code

Some variants/dialects of MLIR:

1. arith
2. func
3. linalg

An example in arith:

```
module {  
  func.func @add_example(%a: i32, %b: i32) -> i32 {  
    %0 = arith.addi %a, %b : i32  
    return %0 : i32  
  }  
}
```

These representations allow for machine-agnostic standardized analyses and optimization passes. Consider optimizations such as constant folding, 1st order algebraic simplification, and dead code elimination. Those are operable on a GENERAL level. Instead of having to understand the AST structures (or bytecode) of some compiler frontend, we can simply operate on them in MLIR.

3 Motivation

The motivation for this project is as follows:

Using Haskell to reason about Python AST objects allows us to leverage Haskell's early type safety, complex logic expressivity, and reliability to directly compile Python code. We can directly work with Python AST nodes, rather than having to go through CPython/other LLVM workflows. This allows direct lowering from high level syntax (Python) to MLIR. Note we can operate directly on function annotations and library calls.

4 Explanation

A more concrete explanation of this approach is: This project will utilize an existing Python AST library (One which takes in Python code and outputs a Python AST). Given an AST, our code will walk it, then call the appropriate mlir-hs functions to create MLIR-IR objects. (linalg or arith.) Finally, the code will print the ensuing IR.

5 Scope/Conclusion

I've scoped this project to act on basic vector ops in native python lists - multiplication, addition, etc. The code will essentially be a mapping between those ops and the appropriate MLIR IR objects. Furthermore, the project will not focus on execution correctness or optimization. Some ways to expand this scope are to act on tensors, actually have it run and test optimization, etc.